30-Day Python Full-Stack Web Development with AI

A practical, project-driven plan covering Python, Django, Django REST Framework (DRF), FastAPI, databases, authentication, deployment, and AI integration. Build one main product across the month: an AI-powered task management SaaS.

Outcome

By Day 30, you will have deployed a full-stack application with Django templates or React, secure REST APIs, JWT authentication, PostgreSQL, background tasks, AI features, tests, Docker, and cloud deployment.

Weekly Phases

Phase	Days	Focus	Main deliverable
Phase 1: Foundations	1–7	Python, Git, HTTP, SQL, virtual environments, Django basics	Working Django app with models, admin, templates
Phase 2: Django + DRF APIs	8–14	DRF, serializers, views, JWT auth, PostgreSQL, testing	Secure CRUD REST API
Phase 3: FastAPI + Advanced Backend	15–21	FastAPI, Pydantic, async, WebSockets, Celery/Redis, file uploads	FastAPI microservice + async features
Phase 4: AI Integration	22–26	OpenAI-compatible APIs, embeddings, RAG, prompt design, vector DB	AI-powered search/chat in the product
Phase 5: Frontend + Deployment	27–30	React or Django templates, Tailwind, Docker, CI/CD, cloud hosting	Deployed full-stack AI SaaS

Day-by-Day Plan

Day 1: Python and tooling setup

Learn: Install Python 3.12+, VS Code, Git, and create a GitHub repository. Learn variables, data types, functions, lists, dictionaries, loops, and error handling.

Build: Build a CLI task tracker.

Day 2: Python OOP and modules

Learn: Classes, inheritance, dataclasses, modules, packages, virtual environments, pip, and requirements.txt.

Build: Refactor the task tracker using classes.

Day 3: Git, HTTP, and web fundamentals

Learn: Git branching, commits, pull requests; HTTP methods, status codes, headers, JSON, REST principles, and browser developer tools.

Build: Create a GitHub repo with a clear README.

Day 4: SQL and databases

Learn: PostgreSQL basics: tables, primary/foreign keys, joins, indexes, transactions, and basic queries. Install PostgreSQL and pgAdmin.

Build: Design a tasks/users database schema.

Day 5: Django project setup

Learn: Create a Django project and app. Understand settings, URLs, views, templates, static files, and the development server.

Build: Run a Django site with a home page.

Day 6: Django models and admin

Learn: Model fields, migrations, QuerySets, relationships, Django admin customization, and environment variables with python-decouple or django-environ.

Build: Create Task, Project, and User-related models.

Day 7: Django templates and forms

Learn: Template inheritance, context, Django forms, ModelForms, validation, messages, and pagination.

Build: Build task create/edit/list pages.

Day 8: DRF fundamentals

Learn: Install DRF. Create serializers, APIView, generics, ViewSets, routers, and JSON responses.

Build: Expose Task CRUD API endpoints.

Day 9: Authentication and permissions

Learn: Token/JWT authentication with SimpleJWT, registration, login, refresh tokens, permissions, and object-level ownership.

Build: Protect task APIs by user.

Day 10: Filtering, search, pagination

Learn: django-filter, search, ordering, pagination, throttling, CORS, and API documentation with drf-spectacular.

Build: Add searchable, paginated task API.

Day 11: Testing the API

Learn: pytest, pytest-django, factory_boy, API tests, coverage, and mocking external services.

Build: Write tests for core endpoints.

Day 12: File uploads and media

Learn: Image/file uploads, validation, storage configuration, serving media in development, and S3-compatible storage concepts.

Build: Allow task attachments.

Day 13: Django signals and emails

Learn: Signals, custom user model, email verification concepts, password reset flow, and transactional email basics.

Build: Send welcome and password-reset emails.

Day 14: Phase review and refactor

Learn: Review code quality, add README/API docs, seed data, fix bugs, and prepare the Django backend for frontend integration.

Build: Clean, documented Django API.

Day 15: FastAPI fundamentals

Learn: Install FastAPI and Uvicorn. Learn path/query parameters, request bodies, Pydantic models, automatic docs, and dependency injection.

Build: Build a small FastAPI service.

Day 16: FastAPI database integration

Learn: SQLAlchemy 2.0 or SQLModel, async database sessions, Alembic migrations, and PostgreSQL connection management.

Build: Create a FastAPI CRUD service.

Day 17: Async Python

Learn: async/await, event loops, async HTTP with httpx, background tasks, and when async helps versus Django.

Build: Add an async external-data endpoint.

Day 18: WebSockets and realtime

Learn: WebSocket concepts, FastAPI WebSocket endpoints, connection handling, and simple realtime updates.

Build: Build a live task-status channel.

Day 19: Background jobs

Learn: Celery or ARQ with Redis. Create delayed and periodic jobs, retries, monitoring, and task queues.

Build: Schedule daily summary emails.

Day 20: Caching and performance

Learn: Redis caching, query optimization, select_related/prefetch_related, N+1 queries, profiling, and rate limiting.

Build: Cache expensive API responses.

Day 21: Security and API design

Learn: OWASP basics, input validation, SQL injection prevention, secrets management, CORS, HTTPS, JWT best practices, and API versioning.

Build: Perform a security checklist review.

Day 22: AI APIs and prompt engineering

Learn: Use an OpenAI-compatible API or local model. Learn chat completions, system prompts, temperature, tokens, streaming, and cost control.

Build: Create an AI task-summary endpoint.

Day 23: Embeddings and semantic search

Learn: Text embeddings, cosine similarity, vector databases such as pgvector/Qdrant/Chroma, and chunking strategies.

Build: Add semantic search over tasks/notes.

Day 24: RAG application

Learn: Retrieval-Augmented Generation: document ingestion, chunking, embeddings, retrieval, context prompts, citations, and evaluation.

Build: Build “ask questions about my tasks/notes.”

Day 25: AI product features

Learn: AI task generation, prioritization, summarization, tagging, and moderation. Add usage limits, fallbacks, logging, and evaluation.

Build: Add three useful AI features.

Day 26: AI reliability and safety

Learn: Handle hallucinations, retries, timeouts, rate limits, PII redaction, prompt injection defenses, and human-review workflows.

Build: Add guardrails and monitoring.

Day 27: Frontend foundation

Learn: Choose React + Vite or Django templates. Build layout, routing, state management, forms, and API client with Axios or fetch.

Build: Create dashboard UI.

Day 28: Connect frontend to APIs

Learn: Authentication flow, protected routes, CRUD screens, optimistic updates, error/loading states, and WebSocket integration.

Build: Complete task management UI.

Day 29: Docker and CI/CD

Learn: Write Dockerfiles and docker-compose for Django/FastAPI, Postgres, Redis, and frontend. Add GitHub Actions for tests and builds.

Build: Containerize the application.

Day 30: Deployment and launch

Learn: Deploy to Render, Railway, Fly.io, AWS, or a VPS. Configure environment variables, HTTPS, database backups, logging, monitoring, and domain.

Build: Launch the deployed product.

Main Capstone: AI Task Management SaaS

Build a multi-user productivity app where users create projects and tasks, attach notes/files, receive AI-generated summaries and suggested next actions, search semantically across their work, and get realtime status updates.

• User registration, login, JWT authentication, and profile management.

• Projects, tasks, labels, comments, attachments, and due dates.

• DRF CRUD APIs with filtering, pagination, permissions, and OpenAPI docs.

• FastAPI service for AI operations, realtime updates, or high-throughput endpoints.

• PostgreSQL for relational data; Redis for cache, queues, and WebSockets support.

• AI features: task summarization, smart prioritization, semantic search, and RAG over notes.

• Dockerized services, automated tests, CI/CD, logging, and cloud deployment.

Daily Routine

Time	Activity
60–90 min	Learn the day’s core concepts from official docs and one high-quality tutorial.
2–3 hours	Code the day’s feature without copying large blocks of code.
30–45 min	Write or update tests and documentation.
20–30 min	Commit to GitHub with a meaningful message and write a short learning note.

Recommended Stack

Layer	Tools
Language	Python 3.12+, Git, GitHub
Django backend	Django 5.x, DRF, SimpleJWT, django-filter, drf-spectacular
FastAPI service	FastAPI, Uvicorn, Pydantic v2, SQLModel/SQLAlchemy, httpx
Data	PostgreSQL, Redis, pgvector or Qdrant/Chroma
AI	OpenAI-compatible API or local model, embeddings, RAG pipeline
Frontend	React + Vite + Tailwind, or Django templates
Ops	Docker, Docker Compose, GitHub Actions, Render/Railway/Fly.io/VPS
Quality	pytest, pytest-django, factory_boy, coverage, ruff, pre-commit

Success Checklist

■ Deployed application accessible through HTTPS.

■ At least 25 meaningful GitHub commits and a professional README.

■ Django/DRF API documented with Swagger or ReDoc.

■ JWT-protected endpoints and role/ownership permissions.

■ At least 20 meaningful automated tests.

■ One FastAPI endpoint or microservice integrated with the main app.

■ One AI feature using prompts, embeddings, or RAG.

■ Docker Compose running all major services locally.

■ CI pipeline running tests before deployment.